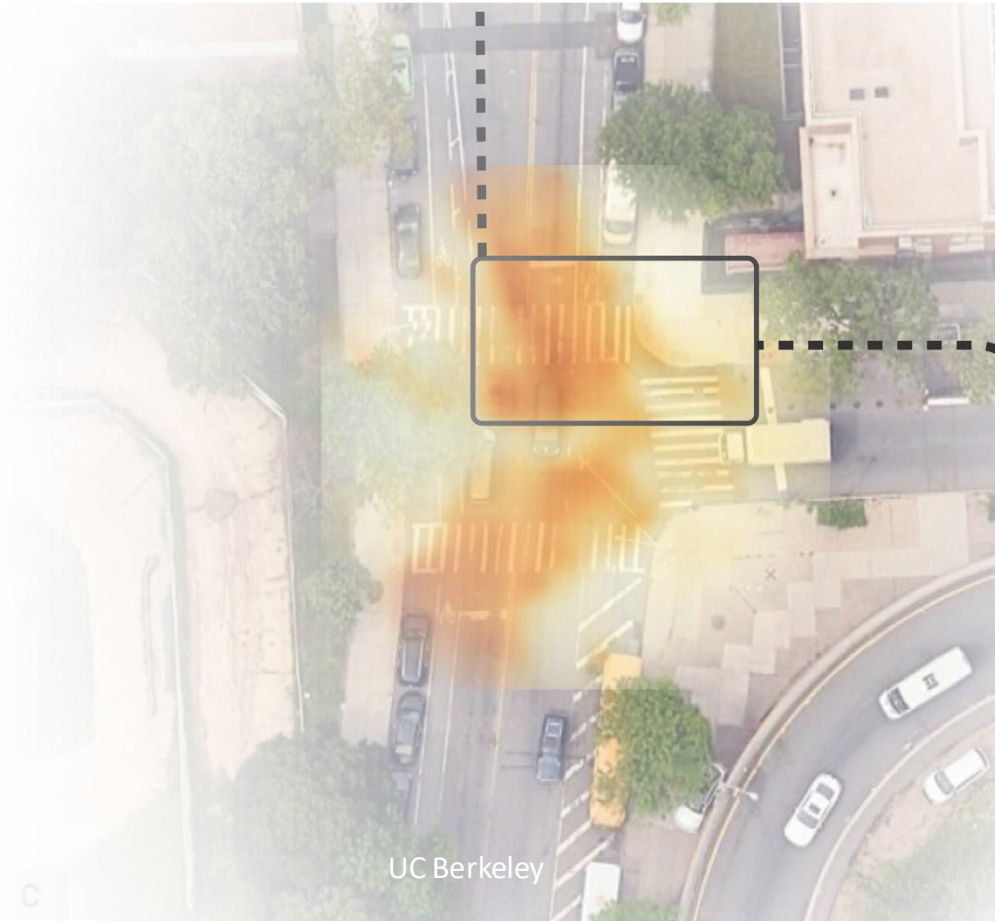
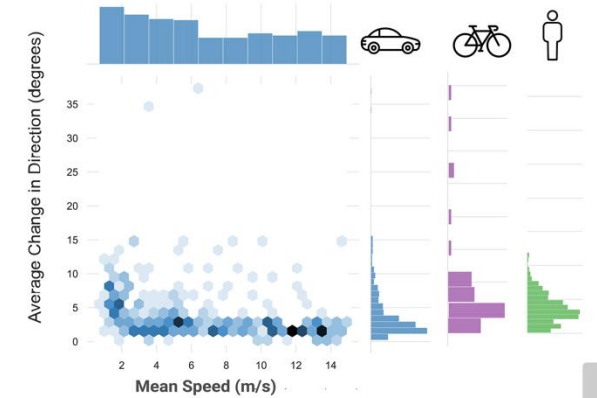
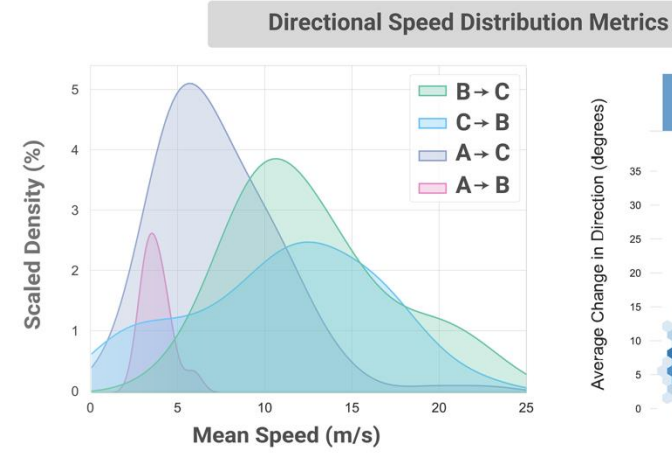
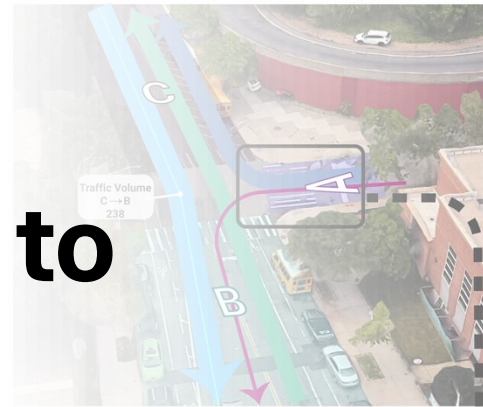


Introduction to Urban Data Analytics

Fall 2026

Maryam Hosseini

City & Regional Planning



UC Berkeley

Python Dictionaries



key-value pairs {key: value} – Mapping type

Dictionary

- Dictionaries are collections of **key-value pairs**.
- Like a real dictionary, we can look up and see some kind of information associated with our keys.
- Unlike sequences which are indexed by a range of numbers, dictionaries are **indexed by keys**, which can be **any immutable type (hashable)**.
- To create a dictionary, we use curly brackets {} and specify key-value pairs in the format {key: value}.
- Dictionaries provide an efficient way to store and retrieve information based on **unique keys**.

Dictionary vs. List

- The key advantage of dictionaries over lists is **the speed of access**.
- Searching for an item in a list can take longer because you have to iterate over the entire list.
- In contrast, dictionaries use a technique called **hashing**, which allows them to find the value associated with a key almost instantly, no matter how large the dictionary is.
- Dictionaries are incredibly versatile and are used extensively in various programming tasks, from storing configurations and settings to handling data in web development and beyond.

Dictionary Keys

- Keys must be unique. No duplicate keys are allowed. If a key is repeated, the last assigned value will overwrite the previous one.
- Keys must be of immutable type.
 - Strings and numbers can always be keys.
 - Tuples can be used as keys if they contain only strings, numbers, or tuples; if a tuple contains any mutable object either directly or indirectly, it cannot be used as a key.
 - You can't use lists as keys, since lists can be modified in place using index assignments, slice assignments, or methods like `append()` and `extend()`.
- To check whether a single key is in the dictionary, use the `in` keyword.

Creating a dictionary

Track the number of items packed for travel:

```
>>> packed = {'dress': 2, 'shirt': 3, 'shoes': 1}
>>> packed.keys() # item names.
dict_keys(['dress', 'shirt', 'shoes'])
>>> packed.values() # quantity of each item packed
dict_values([2, 3, 1])
```

Retrieve the value for any item, use its key:

```
>>> packed['dress']
2
```

Python External Packages

- External packages are like board games' **expansion packs**.
- You need to install the library, and then you can import it to be used
- We use the keyword **import** to import a package into our code.



Pandas



Tabular data cleaning and wrangling

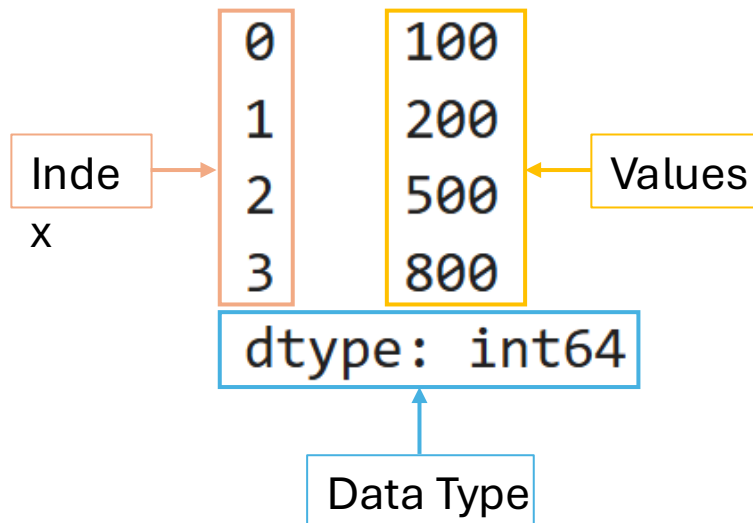
Pandas

- [pandas] is derived from the term "**panel data**", an econometrics term for data sets that include observations over multiple time periods from multiple entities
- Ranked among the most popular and widely used **external Python libraries**
- Works with variety of formats
- By convention, we import it in our code using **import pandas as pd**
- **pd** is now a new **variable** that we created, which points to pandas module and every "methods" it has.
- Pandas main data structures: **Series** and **DataFrame**

Series

A Series is essentially a vertical list, like Excel column

```
my_series = pd.Series([100, 200, 500, 800])  
my_series
```



DataFrame

- A DataFrame is a multi-dimensional table made up of a collection of Series.

Series

	Job_num	Borough	House_num	Street Name	Block	Lot	Bin_num	Job Type	Community - Board	Curb Cut
0	200612337	BRONX	946	LEGGETT AVENUE	2685	50	2005109	A1	202	NaN
1	200612337	BRONX	946	LEGGETT AVENUE	2685	50	2005109	A1	202	NaN
2	200621336	BRONX	1216	BEACH AVENUE	3764	13	2024720	A1	209	NaN
3	200621336	BRONX	1216	BEACH AVENUE	3764	13	2024720	A1	209	NaN
4	200621336	BRONX	1216	BEACH AVENUE	3764	13	2024720	A1	209	NaN
...	...	...	...	...	...	...	...	...	...	...
19591	200621336	BRONX	1216	WELL AVENUE	2623	135	2091318	A2	201	NaN
19592	200621336	BRONX	1216	WELL AVENUE	2623	135	2091318	A2	201	NaN
19593	200621336	BRONX	1216	WELL AVENUE	2623	135	2091318	A2	201	NaN
19594	200621336	BRONX	1216	WELL AVENUE	2623	135	2091318	A2	201	NaN
19595	200621336	BRONX	1216	WELL AVENUE	2623	135	2091318	A2	201	NaN

```
df['Block']
```

```
0      2685
1      2685
2      3764
3      3764
4      2623
...
19591    717
19592    717
19593   1096
19594    541
19595    541
Name: Block, Length: 19596, dtype: int64
```

DataFrame

- A DataFrame is a multi-dimensional table made up of a collection of Series.

column names

columns axis=1

	Job_num	Borough	House_num	Street Name	Block	Lot	Bin_num	Job Type	Community - Board	Curb Cut
0	200612337	BRONX	946	LEGGETT AVENUE	2685	50	2005109	A1	202	NaN
1	200612337	BRONX	946	LEGGETT AVENUE	2685	50	2005109	A1	202	NaN
2	200621336	BRONX	1216	BEACH AVENUE	3764	13	2024720	A1	209	NaN
3	200621336	BRONX	1216	BEACH AVENUE	3764	13	2024720	A1	209	NaN
4	200622424	BRONX	550	CAULDWELL AVENUE	2623	135	2091318	A2	201	NaN

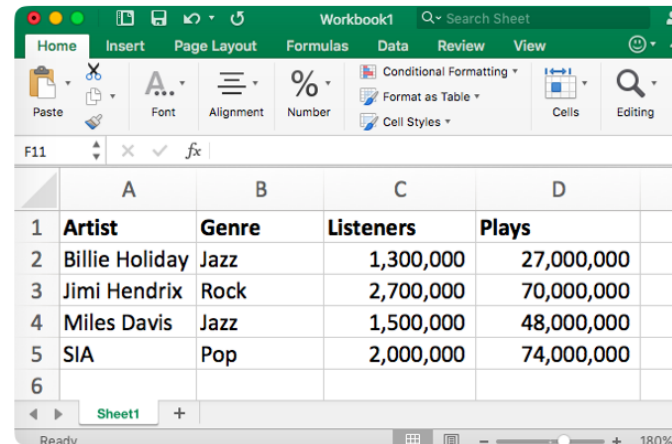
index labels

index axis=0

Missing Value

Reading CSV

music.csv



	Artist	Genre	Listeners	Plays
1	Billie Holiday	Jazz	1,300,000	27,000,000
2	Jimi Hendrix	Rock	2,700,000	70,000,000
3	Miles Davis	Jazz	1,500,000	48,000,000
4	SIA	Pop	2,000,000	74,000,000



`pandas.read_csv('music.csv')`

	Artist	Genre	Listeners	Plays
0	Billie Holiday	Jazz	1,300,000	27,000,000
1	Jimi Hendrix	Rock	2,700,000	70,000,000
2	Miles Davis	Jazz	1,500,000	48,000,000
3	SIA	Pop	2,000,000	74,000,000

<https://jalammar.github.io/gentle-visual-intro-to-data-analysis-python-pandas/>

[Jay Alammar](#)